

Horizon Function Map

Deferred Scope and Known Exclusions

26 July 2026

1 Purpose of this document

This is a record of everything discussed and *deliberately set aside* while designing the function map. It exists so that a decision made for good reasons today is not mistaken for an oversight later, and so that the cost of each exclusion is written down while it is still fresh.

Nothing here is a bug. Each item is either an accepted limitation, a deferral, or an alternative that was evaluated and passed over. Items that remain genuinely undecided are collected separately in Section 8.

Where a number appears, it was measured on real code — Horizon’s own `src/`, the purpose-built demo crates, or published benchmarks — rather than estimated.

2 The frame these exclusions follow from

Four decisions shape everything below.

Functions only.

The map contains functions, their calls, and comments. Data structures are out.

Free functions only.

Methods and `impl` blocks are a separate problem to be handled later.

Valid code only.

The map is updated only from code that compiles.

Batch updates.

The map is rebuilt when a codebase is uploaded to the project, not continuously while code is being written. Between updates the last known-good map is held.

3 Deferred by explicit decision

3.1 Methods and `impl` blocks

Excluded entirely: method declarations inside `impl` blocks, and calls of the form `x.foo()`.

Why it matters: This is the largest single exclusion by volume. Of Horizon’s **1,394 call sites**, **1,053 are method calls** — roughly 75%. Only 266 are the path-form calls the map currently describes. The map will therefore represent a minority of the call activity in a typical Rust codebase.

Resolving them requires knowing the type of the receiver, which needs type inference rather than name lookup. Two sub-cases are known to be unresolvable without a real type checker and would remain excluded even after methods are introduced:

- receivers typed through a generic parameter (`T: Store`),
- receivers behind a trait object or opaque type (`&dyn Store, impl Store`).

A related correctness rule was identified but not implemented: Rust probes inherent methods before trait methods, so when a type has both, the inherent one wins. The demo crate contains exactly this collision.

```
impl Cache {
    pub fn get(&self, key: &str) -> Option<&Item> { ... }
}

impl Store for Cache {
    fn get(&self, key: &str) -> Option<&Item> { ... }
}
```

The spike resolves this correctly today, but by coincidence rather than by rule.

3.2 Data structures

Excluded: `struct`, `enum`, and `trait` definitions; the use of one data structure inside another's definition; and field types.

Why it matters: This was part of the original goal for the project. It is deferred, not abandoned. Note that field types are already extracted internally as an input to receiver typing, so the information is partly available even though it is not represented in the map.

3.3 Invalid and non-compiling code

The map is built only from code that compiles.

Why it matters: Error tolerance was one of the original reasons for choosing `ra_ap_syntax` over `syn`. That capability is retained in the parser but is no longer exercised by the map.

This exclusion is worth more than it costs, because a compiling program cannot contain the two hardest ambiguity cases:

- **Ambiguous names.** Two glob imports supplying the same name are legal until the name is used, at which point the compiler emits `E0659`. Verified against the compiler.
- **Unresolvable names.** A call to a function that does not exist fails with `E0425`.

Because neither can appear in compiling code, path-form call resolution becomes a reading rather than a guess, and the confidence tiers largely collapse. Both categories return the moment error tolerance is reintroduced.

3.4 Live incremental updating

Excluded: updating the map continuously as code is typed.

Why it matters: The original requirement was that the map update as code is being written without re-running the whole analysis. Code mid-edit is usually not compilable, which is incompatible with the valid-code-only rule. The map therefore lags, holding the last known-good state.

3.5 Call categories excluded from the map

Four kinds of call are recognised by the analyser but will not appear as nodes.

External calls.

Calls into `std` or a registry dependency, e.g. `std::cmp::max(1, 2)`. The name and owning crate are known exactly; the exclusion is a deliberate boundary, not a failure.

Why it matters: A function whose body only calls `std` will appear as a childless leaf. It will not be visible that a function is a thin wrapper over, say, `serde_json`.

Indirect calls.

A call through a variable holding a function.

```
fn apply(s: &str, f: fn(&str) -> String) -> String {
    f(s)                                     // 'f' is a parameter, not a function
    name
}
let k = |x: &str| normalize(x);
k(label)                                   // 'k' is a local closure binding
```

Which function runs depends on what the caller passed and may differ per invocation. Determining it requires dataflow analysis.

Functions passed as values.

In `apply(&n, normalize)` the name `normalize` is mentioned but not called.

Why it matters: This relationship vanishes completely today. A function can be passed throughout a codebase and still appear unreferenced in the map.

Unknown names.

Names not found in the index. Cannot occur in compiling code except via macro-generated definitions.

Included by contrast. Calls hidden inside macro arguments *are* in scope and will be recovered heuristically. The parser leaves macro arguments as unstructured token trees, so `mean(v)` inside `format!("{}", mean(v))` is invisible to a normal tree walk. Measured on Horizon this hides **68 calls against 266 visible path calls**, about a quarter of the map. Under the valid-code-only rule the heuristic gains a free validator: recover a call-shaped token sequence, keep it only if the name resolves to exactly one definition.

4 Language features knowingly unhandled

4.1 Configuration-gated duplicate definitions

```
#[cfg(unix)]    fn open() { }
#[cfg(windows)] fn open() { }
```

One path, two real definitions. Which is live depends on a build configuration nobody has chosen. A path lookup therefore returns a *set*, not a single answer.

4.2 Modules absent from the tree spine

The map's containment hierarchy is folder $\rightarrow$ file $\rightarrow$ function. Modules are not nodes, on the working assumption that a module corresponds to a file. That assumption was tested against the compiler and is false in four distinct ways:

Why it matters: Two consequences follow. Inline modules disappear from the spine, so a file may contain two functions of the same name at different module depths — and a call node that references only its declaring *file* cannot distinguish them. Second, a file declared nowhere is not part of the crate at all: a file containing pure syntactic garbage compiles fine as long as no `mod` declaration names it. File discovery by directory globbing is therefore incorrect; discovery must start at the crate roots and follow `mod` declarations.

A further consequence, verified: a `mod` inside a file nests rather than merging, so a file `selfdecl.rs` declaring `pub mod selfdecl` yields the path `crate::selfdecl::selfdecl::inner_fn`.

Case	Module	File
Ordinary	<code>text</code>	<code>text.rs</code>
Inline module	<code>text::case</code>	none — written inside <code>text.rs</code>
Undeclared file	none	<code>orphan.rs</code> — never compiled
<code>#[path]</code>	<code>tidy_name</code>	<code>renamed_on_disk.rs</code>
Crate root	the root itself	<code>lib.rs</code> — contributes no segment

4.3 Textual inclusion

`include!("other.rs")` splices a file's text in with no module boundary. Rare outside code generation, but it means even a complete `mod` walk can miss source that is genuinely compiled.

4.4 Scope-local use declarations

`use` is legal inside functions and modules, not only at file top level. Such imports are currently attributed to the whole file, which is wider than their true scope. Two instances exist in Horizon's own source. The count is reported so the approximation stays visible.

4.5 Glob imports into dependencies

`use some_dependency::*`; cannot be expanded without the dependency's source. Globs into local crates are resolved against the index but held at lower confidence than an explicitly named import.

4.6 Traits imported as `_`

`use SomeTrait as _`; brings a trait into scope for method resolution without binding a nameable identifier, so the trait becomes invisible to the index. Method-related, and therefore doubly deferred.

5 Build-system cases knowingly unhandled

Crate discovery is driven by `cargo metadata`, which was measured at **322 ms on Horizon** and performs no compilation. The following remain unhandled.

Dependency renaming in the manifest.

`foo = { package = "actual-name", path = "..."}` means code says `foo` while the crate is `actual-name` — a manifest-level alias analogous to `use ... as ...`. Identified, not implemented.

`[patch]` and `[replace]`.

These can redirect an apparent registry dependency to a local directory, so a crate can be local without its own entry saying `path`. Same for vendored setups where `.cargo/config.toml` places every dependency on disk.

Optional and feature-gated dependencies.

Presence depends on the selected feature set, which is genuinely undecidable without choosing one.

Target-specific dependencies.

Entries under `[target.'cfg(unix)'.dependencies]` vary by platform.

Development dependencies.

Visible only to tests, examples, and benches, not to the library.

Package name inference.

Currently derived from the directory layout rather than read from the manifest, so a crate whose name differs from its folder, or a multi-crate workspace, falls back to heuristics.

Non-Cargo projects.

Builds driven by `rust-project.json`, Buck, or Bazel are out of scope; everything assumes Cargo.

One case that is handled, recorded because it is unintuitive. A binary in `src/bin/` is a separate crate from the library beside it and must reach it by package name, exactly as an external dependency would — Horizon’s own `src/bin/server.rs` does this with `use horizon::{analyze, scan_pat`. An early version of the import table treated any package-named import as external and silently lost five real edges.

6 Alternatives evaluated and set aside

6.1 rust-analyzer SCIP indexing

`rust-analyzer scip` . emits a complete semantic index — definitions, references, resolved cross-references, hover documentation — using real name resolution and type inference rather than heuristics. Published measurements:

Codebase	Time	Peak memory	Result
ruff	73 s	2.1 GB	646k occurrences, 81% resolved
ripgrep	35 s	1.1 GB	full index
rust-analyzer	35 s	—	full index
mozilla-central	20 min	—	full index
ruff, via live LSP	254 s	15.4 GB	9% resolved, degraded

Unresolved references are `std` and dependencies, marked `EXTERNAL_SYMBOL` — which is the intended external-dependency rule, implemented for free. Coverage includes methods.

Why it matters: This resolves roughly 81% including methods, against the hand-built approach’s roughly 55% of path-form calls with methods excluded. The decision is to build our own for now regardless. The costs of adopting SCIP are a required toolchain (`rustup component add rust-analyzer; cargo 1.95.0` and `rustc 1.95.0` are present, the rust-analyzer component is not), a batch snapshot rather than incremental output, and unverified handling of comments as first-class nodes. The first two objections are weaker under the batch-update model than they would have been under live updating.

6.2 Validity gates not chosen

Three mechanisms for establishing that code compiles were identified. The choice between them remains open (Section 8).

- `cargo check --message-format=json` — the authority. One JSON object per line; the `cargo_metadata` crate parses the stream. Stops before code generation, so a small class of codegen-time diagnostics never appears.
- `rust-analyzer diagnostics <path>` — purpose-built as a gate; exits non-zero if any error is found.

- `rust-analyzer unresolved-references <path>` — narrower and aimed exactly at this problem, reporting `file:line:column: text`. Known false positives around procedural macros unless the workspace is loaded with `all_targets: true`.

Rejected outright: parser errors as a validity gate. Syntactic validity and compilability are almost unrelated. Of the compiler errors produced while testing these rules, only one was syntactic — a dash in a path. Every other one parsed perfectly and failed later:

```
E0433 cannot find module 'text'          <- parsed fine
E0425 cannot find function 'get'         <- parsed fine
E0603 module 'secret' is private        <- parsed fine
E0659 'get' is ambiguous                 <- parsed fine
E0308 mismatched types                  <- parsed fine
```

7 Known weaknesses in what already exists

Correctness is unmeasured.

Every figure reported so far is *coverage* — how many calls resolved. The false-positive rate is unknown. A confidently wrong edge is worse for the map than an honest refusal, and there is currently no estimate of how many exist.

Import refutation is unproven.

Logic exists to suppress an edge when an import names an external dependency, but across all three test corpora it never fired.

Re-export chains are truncated.

Followed to four hops, then abandoned.

Edition is hardcoded.

The parser is invoked with `Edition::CURRENT`. Horizon is edition 2021; the Cellular Automata crates are edition 2024. Editions change parsing behaviour, and `cargo metadata` reports the true per-target value.

File discovery is by directory glob.

As established above, this is the wrong mechanism.

8 Undecided

These were raised and not resolved. They are not exclusions.

1. **Where comments attach.** A file's children are functions, and a function's children may be comments. Module-level documentation (`//!` at the top of a file) and free-floating comments attached to nothing therefore have no place in the structure.
2. **Crates versus folders in the spine.** The spine is folder $\rightarrow$ file, but the meaningful unit is the crate, and crates do not nest like folders. `C:\Users\kouat\Research\Cellular Automata` contains two independent crates with no manifest at the repository root; `src/bin/*.rs` files are separate crate roots sharing a folder; and undeclared files belong to no crate at all. Whether crates appear as nodes is unsettled.
3. **What a call node references.** Referencing the declaring *file* is ambiguous when a file holds two functions of the same name at different module depths. Referencing the *function* object instead gives exact identity, with the file still reachable by walking up the containment tree. Proposed, not confirmed.

4. **Which validity gate to use** (Section 6.2).
5. **Threading the real edition through** from `cargo metadata` to the parser.

9 Original goals not addressed today

- **Multi-language support.** The project is intended to work on arbitrary codebases; everything designed today is Rust-specific.
- **Scale.** A 50,000-file target was discussed previously. Current extraction throughput is roughly 1.2 MB/s single-core, and no parallel or hierarchical linking scheme has been built.
- **Identity under re-export.** One definition can be reachable by several paths, so identity should be a canonical definition site plus an alias table. Import aliases are handled; the general case is not.